



REPORTE DE PRÁCTICA

Práctica No. 21

Conexión móvil API y Navigation Stack

UPDATE • DELETE

Alumno	Joan Osvaldo Higuera Gallardo
Grupo	S201
Docente	Isc. Iván Isay Guerra López
Fecha	30 de julio de 2026

Universidad Politécnica de Querétaro

1. Objetivo

Implementar navegación Stack en el proyecto de usuarios y completar las operaciones Update y Delete de la base de datos desde la aplicación móvil.

2. Alcance solicitado

- Agregar la opción **Ver detalle** en cada usuario de la lista.
- Navegar a una pantalla Detalle del usuario mediante Navigation Stack.
- Abrir un formulario de actualización con nombre y edad precargados.
- Guardar los cambios mediante una petición PUT a la API.
- Solicitar confirmación positiva en un modal antes de eliminar.
- Eliminar mediante DELETE y actualizar la lista.

3. Tecnologías utilizadas

Componente	Tecnología	Responsabilidad
Aplicación	React Native + Expo 54	Interfaz y validaciones
Navegación	Expo Router 6	Tabs y Stack
API	FastAPI + SQLAlchemy	Endpoints REST
Base de datos	PostgreSQL 15	Persistencia con Docker
Seguridad	HTTP Basic	Protección de PUT y DELETE

4. Desarrollo de la práctica

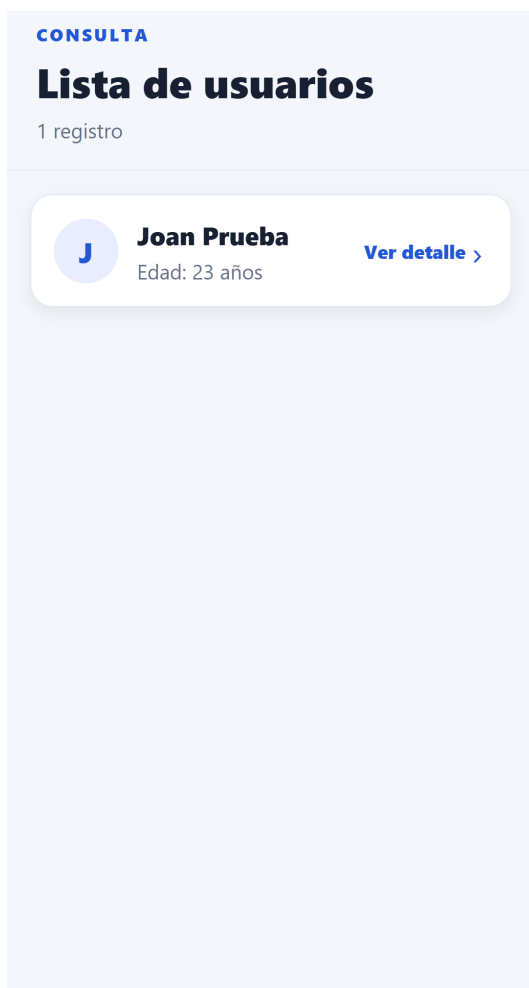
1. Se revisó el proyecto anterior y el contrato actual de la API.
2. Se centralizó el consumo HTTP en **services/usuariosApi.js**.
3. La consulta recibió la acción Ver detalle y recarga al recuperar el foco.
4. Se agregó la ruta detalle-usuario dentro del Stack principal.
5. La API recibió un endpoint GET por identificador.
6. Se creó el formulario con nombre y edad precargados.
7. PUT y DELETE incluyen las credenciales HTTP Basic requeridas.
8. La eliminación se protegió con un modal de confirmación.
9. Después de actualizar se refresca el detalle; después de eliminar se regresa a la lista.

5. Archivos principales

Archivo	Función
app/_layout.js	Define el Stack de detalle y actualización.
screens/ConsultaUsuariosScreen.js	Lista usuarios y abre Ver detalle.
screens/DetalleUsuarioScreen.js	Muestra datos y controla el modal.
screens/ActualizarUsuarioScreen.js	Precarga y envía los cambios.
services/usuariosApi.js	Concentra GET, POST, PUT y DELETE.
miAPI/app/routers/usuarios.py	Expone los endpoints del CRUD.

6. Evidencias de funcionamiento

La prueba se ejecutó en un navegador con emulación móvil, consumiendo la API real levantada localmente. El usuario aparece en la consulta y puede abrirse mediante Navigation Stack.



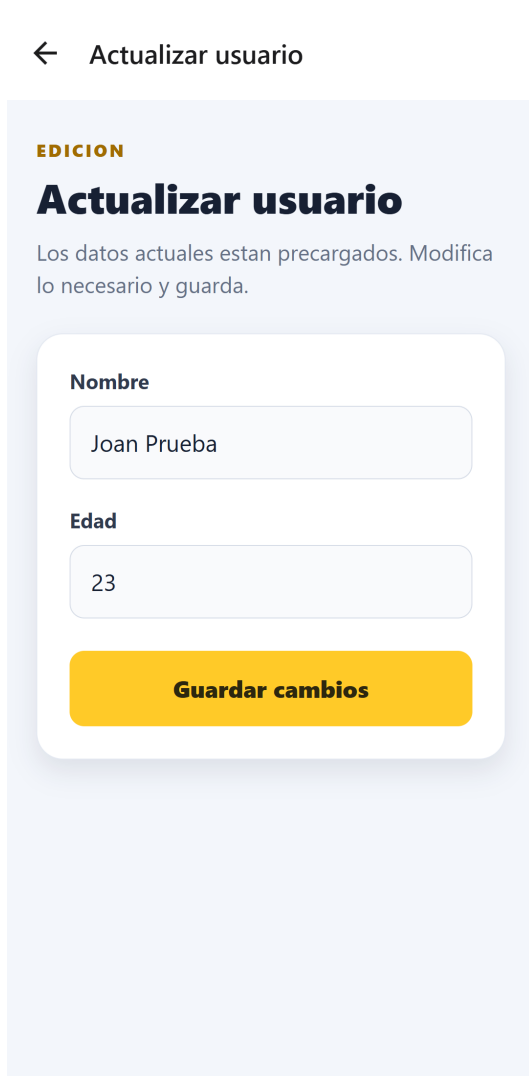
Captura 1. Lista con Ver detalle.



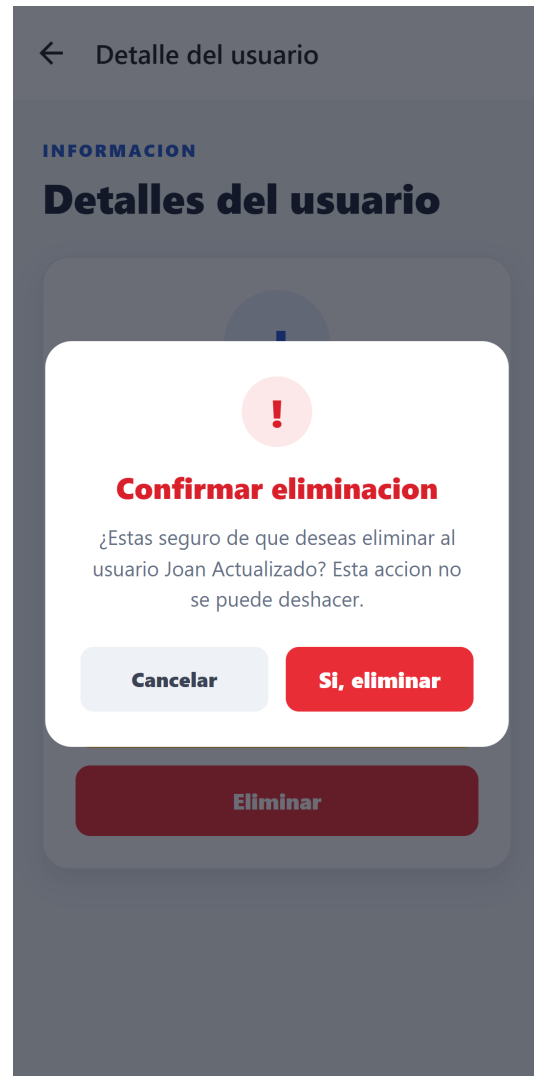
Captura 2. Detalles del usuario.

7. Actualización y eliminación

El formulario recupera el registro por identificador y precarga sus datos. La eliminación requiere una confirmación explícita desde el modal.



Captura 3. Formulario precargado.



Captura 4. Confirmación antes de DELETE.

8. Validación realizada

Prueba	Resultado	Estado
Registrar usuario	POST 201	Correcto
Consultar lista	GET 200	Correcto
Abrir detalle	GET /{id} 200	Correcto
Guardar actualización	PUT 200	Correcto
Cancelar eliminación	El registro permanece	Correcto
Confirmar eliminación	DELETE 200	Correcto
Recargar consulta	GET 200 y lista actualizada	Correcto
Compilar aplicación web	Exportación completada	Correcto

9. Conclusión

Se completó el CRUD del proyecto Usuarios desde la aplicación móvil. Expo Router permitió conservar las pestañas y agregar un flujo Stack para consultar y editar registros. La actualización refleja los cambios al regresar al detalle y la eliminación está protegida por una confirmación, evitando borrados accidentales. Las respuestas HTTP y el recorrido visual confirmaron que el flujo funciona de principio a fin.

10. Repositorio

<https://github.com/Josved/Practica20-Conexion-Movil-API>

Entrega adicional: grabación de pantalla del teléfono mostrando el flujo completo. El video debe comenzar mostrando el fondo de pantalla.